

RASPBERRY PI TELEGRAM BOT GUIDE

English edition // beginner-friendly reliable bot node

Build

- Telegram BotFather
- Python bot
- SQLite notes
- PuTTY workflow

Operate

- systemd autostart
- restart after crash
- reboot recovery
- backup and restore

Light corporate cyberpunk edition // v1.2 // copy/paste-first tutorial for beginners.

WHAT THIS GUIDE BUILDS

Mission snapshot

A reliable Telegram bot node on a Raspberry Pi: BotFather token, Python bot, SQLite notes, backup files, PuTTY workflow, systemd autostart, reboot recovery and optional local AI. The guide is written for beginners and uses complete copy/paste blocks.

Layer	Choice	Why
Telegram	BotFather token	Official bot creation path
Host	Raspberry Pi 3/4/5	Low-power always-on Linux node
Runtime	Python venv	Clean dependency isolation
Service	systemd	Autostart and restart after crashes
Storage	SQLite	Simple local database
AI optional	Ollama	Only after base bot is stable

COPY/PASTE WITHOUT LOSING CONTROL

How to use this guide

Work in stages. Paste one block, read the output, then continue. For PuTTY, set the window to stay open after errors.

Rule 1

If a command prints an error, stop and check logs.

Rule 2

Never paste your real token into GitHub. Store it in .env only.

Rule 3

For changes, back up first, compile, then restart the service.

PuTTY SETUP

```
# Windows: download PuTTY from the official PuTTY website.
# Start PuTTY, then fill in:
Host Name: raspberrypi.local
Port: 22
Connection type: SSH

# Recommended PuTTY setting:
Window -> Behaviour -> Close window on exit -> Never
Session -> Save
```

PI 3 / PI 4 / PI 5 EXPECTATIONS

Hardware map

A plain Telegram bot is light. Local AI is the heavy part. Pick hardware for the heaviest thing the bot will do.

Host	Bot	Local AI	Comment
Pi 3 / 3 B+	Yes	No LLM	Commands, notes, sensors
Pi 4 2GB	Yes	No LLM / 0.5B only	Service node
Pi 4 4GB	Yes	0.5B Q4, 1.5B slow	Short prompts
Pi 4 8GB	Yes	0.5B-1.5B, 3B experimental	Best Pi 4
Pi 5 4GB	Strong	0.5B-1.5B	Fast bot
Pi 5 8GB	Recommended	1.5B-3B	Best normal choice
Pi 5 16GB	Best Pi	3B-4B, 7B slow	Most RAM

OLLAMA AND HUIHUI EXAMPLES

Local AI sizing

A Telegram bot should answer quickly. Use local AI behind explicit commands such as /brief or /ai.

Model size	Pi 4 4GB	Pi 4 8GB	Pi 5 8GB	Pi 5 16GB
0.5B Q4	Usable	Usable	Smooth	Smooth
1.5B Q4	Slow	Usable	Good	Good
3B Q4	Avoid	Slow	Usable	Good
4B Q4/Q5	Avoid	Avoid/slow	Borderline	Usable
7B Q4	Avoid	Avoid	Slow	Possible

OPTIONAL OLLAMA SMOKE TESTS

```
# Optional. Do this only after the base bot is reliable.
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
ollama run qwen2.5:0.5b
ollama run qwen2.5:1.5b
ollama run qwen2.5:3b

# Huihui examples - check current tags on ollama.com/huihui_ai first.
ollama run huihui_ai/phi4-mini-abliterated
ollama run huihui_ai/qwen3-abliterated
```

Check current model tags on Ollama before publishing exact names.

CREATE THE TELEGRAM BOT

BotFather setup

Open Telegram, search for @BotFather, create a new bot and copy the token into .env later.

BOTFATHER COMMANDS

```
/newbot

# Display name example:
Raspi Node Bot

# Username example. It must end in bot:
raspi_node_demo_bot

/setcommands
start - boot message
status - service health
last - show recent notes
files - export backup files
help - command list
```

HEADLESS SERVER BASICS

Flash Raspberry Pi OS

Use Raspberry Pi Imager. Raspberry Pi OS Lite 64-bit is enough for a bot server.

FIRST LOGIN

```
# Windows PowerShell / macOS Terminal / Linux Terminal:
ssh pi@raspberrypi.local

# If .local does not resolve, use the IP address from your router:
ssh pi@192.168.1.50
```

CONFIRM THE PI IS REACHABLE

First node check

After SSH login, run a small check.

IDENTITY CHECK

```
whoami
hostname
hostname -I
uname -a
uptime
```

PYTHON, VENV AND PROJECT FOLDER

Install base stack

This creates ~/telegram-bot and installs Python dependencies.

INSTALL PACKAGES AND VENV

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y python3-venv python3-pip sqlite3 git curl nano ca-certificates

mkdir -p ~/telegram-bot
cd ~/telegram-bot
python3 -m venv venv
./venv/bin/python -m pip install --upgrade pip
./venv/bin/pip install python-telegram-bot==22.8
```

TOKEN WITHOUT HARD-CODING

Create .env

The bot reads BOT_TOKEN from .env. This keeps secrets out of bot.py.

CREATE .ENV

```
cd ~/telegram-bot
cat > .env <<'ENVFILE'
BOT_TOKEN=PASTE_BOTFATHER_TOKEN_HERE
BOT_NAME=raspi-node
ENVFILE

chmod 600 .env
ls -l .env
```

Generic bot code

Generic code: stores text immediately and does not call AI during normal intake.

WRITE BOT.PY

```
cd ~/telegram-bot
cat > bot.py <<'BOTPY'
import json
import os
import re
import sqlite3
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo

from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, ContextTypes, filters

BASE = Path(__file__).resolve().parent
DB_FILE = BASE / "bot.db"
ENV_FILE = BASE / ".env"
BACKUP_DIR = BASE / "backups"
TZ = ZoneInfo("Europe/Berlin")

def load_env():
    if not ENV_FILE.exists():
        return
    for line in ENV_FILE.read_text(encoding="utf-8", errors="ignore").splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))

def now_local():
    return datetime.now(TZ)

def clean_text(text):
    text = str(text or "").strip()
    text = re.sub(r"\s+", " ", text)
    for before, after in {" ": " ", ".": ".", " !": "!", " ?": "?"}.items():
        text = text.replace(before, after)
    return text.strip()

def db():
    con = sqlite3.connect(DB_FILE)
    con.row_factory = sqlite3.Row
    return con

def init_db():
    con = db()
    cur = con.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            created_at TEXT NOT NULL,
            text TEXT NOT NULL
        )
    """)
    con.commit()
    con.close()

def add_note(text):
    text = clean_text(text)
    if not text:
        raise ValueError("empty note")
    con = db()
    cur = con.cursor()
    cur.execute("INSERT INTO notes (created_at, text) VALUES (?, ?)",
        (now_local().isoformat(timespec="seconds"), text))
    note_id = cur.lastrowid
    con.commit()
    con.close()
    return note_id

def get_last(limit=10):
    con = db()
    rows = con.execute("SELECT id, created_at, text FROM notes ORDER BY id DESC LIMIT ?",
        (limit,)).fetchall()
    con.close()
    return list(reversed(rows))

def format_notes(rows):
    if not rows:
```

```

        return "VAULT.EMPTY // no notes stored"
    return "\n".join(f"- #{r['id']} {r['text']}" for r in rows)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "[RASPI.BOT]\nNODE ONLINE // send normal text to store a note\n\n"
        "/status - health check\n/last - latest notes\n/files - export backups\n/help - command list"
    )

async def status(update: Update, context: ContextTypes.DEFAULT_TYPE):
    con = db()
    total = con.execute("SELECT COUNT(*) AS c FROM notes").fetchone()["c"]
    con.close()
    await update.message.reply_text(f"STATUS // ONLINE\nSTORED // {total} notes")

async def last(update: Update, context: ContextTypes.DEFAULT_TYPE):
    limit = 10
    if context.args:
        try:
            limit = max(1, min(50, int(context.args[0])))
        except ValueError:
            limit = 10
    await update.message.reply_text("[LATEST]\n" + format_notes(get_last(limit)))

async def files(update: Update, context: ContextTypes.DEFAULT_TYPE):
    BACKUP_DIR.mkdir(exist_ok=True)
    stamp = now.local().strftime("%Y%m%d-%H%M%S")
    txt = BACKUP_DIR / f"notes-{stamp}.txt"
    jsonl = BACKUP_DIR / f"notes-{stamp}.jsonl"
    rows = get_last(100000)
    txt.write_text(format_notes(rows) + "\n", encoding="utf-8")
    with jsonl.open("w", encoding="utf-8") as f:
        for r in rows:
            f.write(json.dumps(dict(r), ensure_ascii=False) + "\n")
    await update.message.reply_text("FILE.NODE // exporting backup files")
    await update.message.reply_document(document=txt.open("rb"), filename=txt.name)
    await update.message.reply_document(document=jsonl.open("rb"), filename=jsonl.name)

async def help_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await start(update, context)

async def store_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = clean_text(update.message.text)
    if not text:
        await update.message.reply_text("INTAKE.REJECTED // empty message")
        return
    note_id = add_note(text)
    await update.message.reply_text(f"[LOGGED]\n- #{note_id} {text}")

def main():
    load_env()
    token = os.environ.get("BOT_TOKEN", "").strip()
    if not token:
        raise RuntimeError("BOT_TOKEN missing in .env")
    init_db()
    app = Application.builder().token(token).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", help_cmd))
    app.add_handler(CommandHandler("status", status))
    app.add_handler(CommandHandler("last", last))
    app.add_handler(CommandHandler("files", files))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, store_text))
    app.run_polling(allowed_updates=Update.ALL_TYPES, drop_pending_updates=True)

if __name__ == "__main__":
    main()
BOTPY

```

BEFORE SYSTEMD

Manual run test

Run the bot manually first. Stop with Ctrl+C after the test works.

MANUAL TEST

```
cd ~/telegram-bot
./venv/bin/python -m py_compile bot.py
./venv/bin/python bot.py

# Stop the manual run with Ctrl+C after the Telegram test works.
```

AUTOSTART AFTER REBOOT AND POWER LOSS

Install as service

systemd starts the bot at boot and restarts it after crashes.

CREATE AND ENABLE SERVICE

```
sudo tee /etc/systemd/system/telegram-bot.service > /dev/null <<'SERVICE'
[Unit]
Description=Raspberry Pi Telegram Bot
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/telegram-bot
EnvironmentFile=/home/pi/telegram-bot/.env
ExecStart=/home/pi/telegram-bot/venv/bin/python /home/pi/telegram-bot/bot.py
Restart=always
RestartSec=10
TimeoutStartSec=60

[Install]
WantedBy=multi-user.target
SERVICE

sudo systemctl daemon-reload
sudo systemctl enable telegram-bot
sudo systemctl start telegram-bot
systemctl status telegram-bot --no-pager -l
```

If your username is not pi, replace User=pi and /home/pi.

PROVE THAT IT COMES BACK

Power-loss and reboot test

A service is only trusted after a reboot test.

REBOOT RECOVERY TEST

```
# Test normal reboot recovery:
sudo reboot

# Reconnect after 30-90 seconds:
ssh pi@raspberrypi.local

# Check whether the bot came back automatically:
systemctl status telegram-bot --no-pager -l
journalctl -u telegram-bot -n 80 --no-pager
```


WHEN SOMETHING BREAKS

Logs and recovery

Do not guess. Use status and journalctl.

SERVICE LOGS

```
systemctl status telegram-bot --no-pager -l
journalctl -u telegram-bot -n 120 --no-pager
journalctl -u telegram-bot -f
```

FOR BEGINNERS WHO DO NOT WANT NANO

Edit safely by copy/paste

Safe loop: stop service, backup, replace or patch, compile, restart.

SAFE FULL-FILE REPLACEMENT

```
cd ~/telegram-bot
sudo systemctl stop telegram-bot
cp bot.py bot.py.bak-$(date +%Y%m%d-%H%M%S)
cat > bot.py <<'BOTPY'
# paste the complete new bot.py here
BOTPY
./venv/bin/python -m py_compile bot.py
sudo systemctl start telegram-bot
systemctl status telegram-bot --no-pager -l
```

CHANGE TEXT OR ADD A COMMAND

Small code changes

Use small patch blocks for small changes.

Small patch

Use for wording changes.

Full replacement

Safer for larger changes.

PATCH REPLY TEXT

```
cd ~/telegram-bot
cp bot.py bot.py.bak-text-change-$(date +%Y%m%d-%H%M%S)
python3 - <<'PY'
from pathlib import Path
p = Path('bot.py')
s = p.read_text(encoding='utf-8')
s = s.replace('NODE ONLINE // send normal text to store a note', 'NODE ONLINE // intake ready')
p.write_text(s, encoding='utf-8')
PY
./venv/bin/python -m py_compile bot.py && sudo systemctl restart telegram-bot
```

MINIMAL COMMAND EXTENSION

Add a command pattern

Every command needs an async function and one CommandHandler registration.

ADD /PING

```
# Add this function above main():
async def ping(update, context):
    await update.message.reply_text("PONG // node alive")

# Then add this handler inside main(), near the other handlers:
app.add_handler(CommandHandler("ping", ping))

# Check and restart:
./venv/bin/python -m py_compile bot.py && sudo systemctl restart telegram-bot
```

DO NOT TRUST ONE SD CARD

Backups and restore

Back up the project folder and database.

BACKUP AND RESTORE FOLDER

```
# Create a local backup archive:
cd ~
tar -czf telegram-bot-backup-$(date +%Y%m%d-%H%M%S).tar.gz telegram-bot
ls -lh telegram-bot-backup-*.tar.gz | tail

# Restore an archive:
cd ~
sudo systemctl stop telegram-bot
mv telegram-bot telegram-bot.bad-$(date +%Y%m%d-%H%M%S)
tar -xzf telegram-bot-backup-YYYYMMDD-HHMMSS.tar.gz
sudo systemctl start telegram-bot
```

SYMPTOMS TO COMMANDS

Troubleshooting map

Use this as the first diagnostic page.

Symptom	Likely cause	Command
Bot silent	Service down	<code>systemctl status telegram-bot</code>
Token error	Bad .env	check .env carefully
Delayed replies	Slow handler/network	<code>journalctl -u telegram-bot -f</code>
PuTTY closes	Session exits	Close window on exit = Never
Duplicate replies	Two bots running	<code>ps -ef grep python</code>
Database error	Schema mismatch	<code>sqlite3 bot.db .schema</code>
No autostart	Service not enabled	<code>systemctl is-enabled telegram-bot</code>

FULL HEALTH CHECK

```
cd ~/telegram-bot
./venv/bin/python -m py_compile bot.py
systemctl is-enabled telegram-bot
systemctl is-active telegram-bot
systemctl status telegram-bot --no-pager -l | head -n 60
journalctl -u telegram-bot -n 80 --no-pager
```

BEFORE PUBLISHING OR LEAVING IT ONLINE

Security checklist

Telegram bot tokens are credentials.

PERMISSION CHECK

```
cd ~/telegram-bot
ls -lah
ls -l .env bot.py bot.db 2>/dev/null
chmod 600 .env
chmod 644 bot.py
```

FAQ

Do I need a Raspberry Pi 5?

No. A Pi 3 or Pi 4 is enough for a simple Telegram bot. Use Pi 5 mainly for heavier automation or local AI tests.

Will the bot restart after a power cut?

Yes, if the systemd service is enabled with systemctl enable and the Pi boots normally.

Why use systemd?

It starts the bot at boot, restarts it after crashes and gives you journalctl logs.

What is BotFather?

BotFather is Telegram's official bot used to create and manage bot accounts.

Where do I put the token?

Put it in .env, never directly in bot.py and never in GitHub.

Can I use PuTTY?

Yes. PuTTY is ideal on Windows. Set Close window on exit to Never so errors stay visible.

Why does PuTTY close after commands?

Usually because the shell exits. Change PuTTY behavior or avoid command blocks with exit statements.

Can I copy/paste the blocks directly?

Yes. Replace placeholders like BOT_TOKEN and username first.

Can I edit the bot without nano?

Yes. Use heredoc replacement blocks or Python patch snippets.

What if the bot stops replying?

Check systemctl status and journalctl, then restart the service.

What if /today or /last is delayed?

Avoid slow AI in the normal message handler. Store first, process later.

Can local AI classify every note?

Technically yes, but on Raspberry Pi it can make the bot feel broken. Use explicit commands.

Which database is used?

SQLite. It is simple, local and needs no server.

Can I expose the bot publicly?

The Telegram bot is reachable through Telegram. Do not expose SSH to the open internet unless you know what you are doing.

Can I run multiple bots?

Yes, but use separate directories, services and BotFather tokens.

What if two processes use the same token?

Telegram polling conflicts can happen. Stop duplicate services.

How do I see running Python bots?

Use ps -ef | grep -i python and systemctl list-units.

How do I update code safely?

Stop service, backup file, replace code, py_compile, restart.

What is drop_pending_updates?

It discards old queued Telegram updates when the bot starts.

Can the bot send files?

Yes. The example /files command exports backups as TXT and JSONL.

Can I keep backups on the Pi?

Yes, but copy them off-device too. SD cards fail.

Should I use an SSD?

For serious always-on use, SSD is more robust than cheap microSD.

Is Wi-Fi okay?

Yes for most bots. Ethernet is better for reliability.

Does network-online.target guarantee internet forever?

No. It only delays startup until network is considered online. The bot must handle later drops.

What if DNS or Telegram is unreachable?

The service restarts and the bot should recover when network returns.

Which Pi is best for local AI?

Pi 5 8GB or 16GB. Still expect CPU-bound speeds.

Which models are realistic?

0.5B to 3B quantized models are the practical range. 4B can be usable on Pi 5 16GB. 7B is not fluid.

Can I use Huihui models?

Yes, but check current tags and run small tests. Abliterated models should be private and supervised.

Do I need Docker?

No. This guide uses native Python and systemd.

Can I use a desktop Raspberry Pi OS?

Yes, but Lite wastes fewer resources.

How do I change the bot name?

Change it in BotFather and adjust text in bot.py if needed.

How do I add a command?

Write an async function and register it with CommandHandler.

Why do I run py_compile?

It catches syntax errors before restarting the service.

What license should I use?

If everyone may reuse everything, CC0 is the simplest public-domain-style choice.

Can I publish this repo?

Yes, if it contains only examples and no tokens, databases, chat IDs or personal logs.

What should I test before calling it finished?

Manual run, Telegram reply, systemd start, reboot recovery, logs and backup export.

Sources and references

Telegram Bot tutorial / BotFather

<https://core.telegram.org/bots/tutorial>

BotFather profile and management entry

<https://telegram.me/BotFather>

python-telegram-bot documentation

<https://docs.python-telegram-bot.org/en/stable/>

Raspberry Pi SSH remote access documentation

<https://www.raspberrypi.com/documentation/configuration/ssh/remote-access.html>

Raspberry Pi 3 Model B+ product page

<https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>

Raspberry Pi 4 Model B specifications

<https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>

Raspberry Pi 5 product page

<https://www.raspberrypi.com/products/raspberry-pi-5/>

systemd network-online guidance

https://systemd.io/NETWORK_ONLINE/

Ollama documentation

<https://docs.ollama.com/>

Ollama Huihui phi4-mini abliterated model page

https://ollama.com/huihui_ai/phi4-mini-abliterated

Ollama Huihui qwen3 abliterated model page

https://ollama.com/huihui_ai/qwen3-abliterated